

Deep Learning (CAI-466)

Objectives:

- Policies
- State-value function
- Action-value function

The objective of a decision-making agent

At first, it seems the agent's goal is to find a sequence of actions that will maximize the return: the sum of rewards (discounted or undiscounted—depending on the value of gamma) during an episode or the entire life of the agent, depending on the task.

Let me introduce a new environment to explain these concepts more concretely, the Slippery Walk Five (SWF) environment.

The Slippery Walk Five (SWF) is a one-row grid-world environment (a walk), that's stochastic, similar to the Frozen Lake, and it has only five non-terminal states (seven total if we count the two terminal).

The Slippery Walk Five environment

(1) This environment is stochastic, and even if the agent selects the Right action, there's a chance it goes left!



- (2) 50% action is success.
- (3) 33.33% stays in place.
- (4) 16.66% goes backward.

The agent starts in *S*, *H* is a hole, *G* is the goal and provides a +1 reward.

(1) The return is the sum of rewards encounter from step t , until the final step T .

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T$$

(2) As I mentioned in the previous chapter, we can combine the return and time using the discount factor, gamma. This is then the discounted return, which prioritizes early rewards.

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T$$

(3) We can simplify the equation and have a more general equation, such as this one.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$G_t = R_{t+1} + \gamma G_{t+1}$$

(4) And stare at this recursive definition of G for a while.

You can think of returns as backward looking—“how much you got” from a past time step; but another way to look at it is as a “reward-to-go”—basically, forward looking. For example, imagine an episode in the SWF environment went this way: State 3 (0 reward), state 4 (0 reward), state 5 (0 reward), state 4 (0 reward), state 5 (0 reward), state 6 (+1 reward). We can shorten it: 3/0, 4/0, 5/0, 4/0, 5/0, 6/1. What’s the return of this trajectory/episode?

Well, if we use discounting the math would work out this way.

Discounted return in the slippery walk five environment

$$G_0 = 1 * 0 + 0.99 * 0 + 0.9801 * 0 + 0.9702 * 0 + 0.9605 * 0 + 0.9509 * 1$$

(1) Calculating the return at time step $t=0$

(2) This is the reward obtained at time step $t+1(0)$ discounted by gamma (0.99).

(3) Reward at $t+2$, discounted by gamma raised to the power 1

(4) Discounted reward at $t+3$

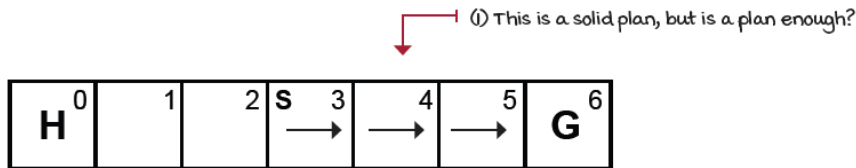
(5) And soon...

(6) ... this is the discounted reward at time step T (final step).

If we don’t use discounting, well, the return would be 1 for this trajectory and all trajectories that end in the right-most cell, state 6, and 0 for all trajectories that terminate in the left-most cell, state 0.

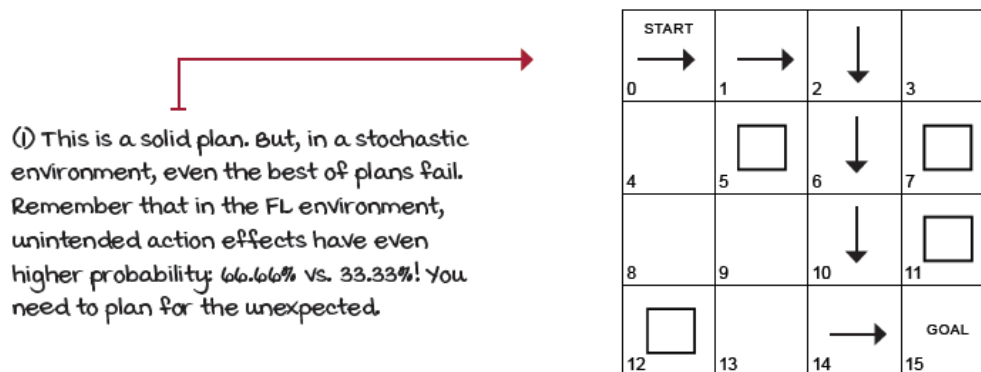
In the SWF environment, it's evident that going right is the best thing to do. It may seem, therefore, that all the agent must find is something called a *plan*—that is, a sequence of actions from the START state to the GOAL state. But this doesn't always work.

A solid plan in the SWF environment



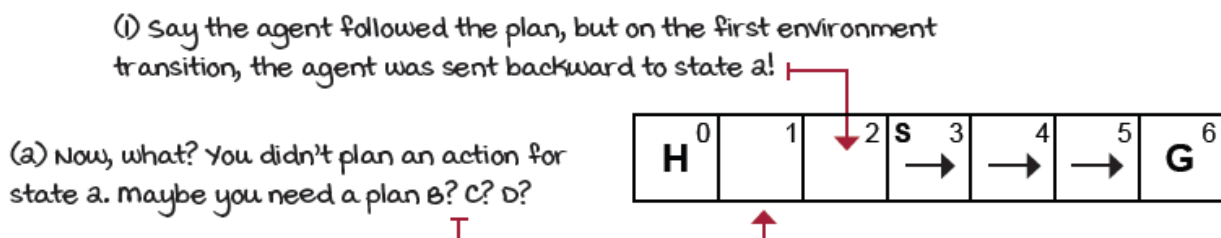
In the FL environment, a plan would look like the following.

A solid plan in the FL environment



But this isn't enough! The problem with plans is they don't account for stochasticity in environments, and both the SWF and FL are stochastic; actions taken won't always work the way we intend. What would happen if, due to the environment's stochasticity, our agent lands on a cell not covered by our plan?

A possible hole in our plan



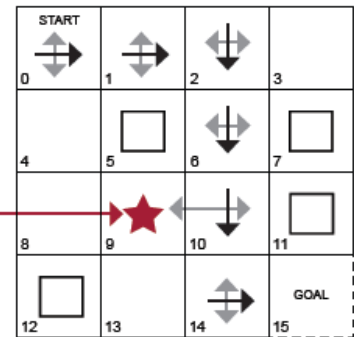
Same happens in the FL environment.

Plans aren't enough in stochastic environments

(1) Here I'm showing the action and the possible action effects. Notice that there's a 66.66% chance that an unintended consequence happens!

(2) Imagine that the agent is following the plan, but in state 10, the agent is sent to state 9, even if it selected the down action, as it apparently is the right thing to do.

(3) What we need is a plan for every possible state, a universal plan, a policy.



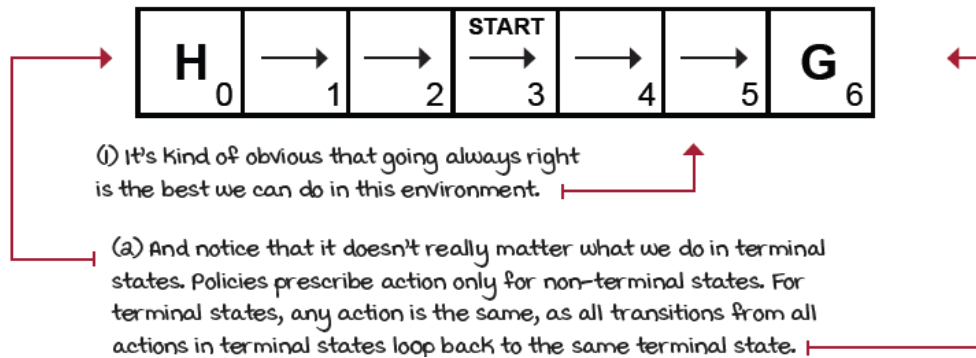
What the agent needs to come up with is called a *policy*. Policies are universal plans; policies cover all possible states. We need to plan for every possible state. Policies can be stochastic or deterministic: the policy can return action-probability distributions or single actions for a given state (or observation). For now, we're working with deterministic policies, which is a lookup table that maps actions to states.

In the SWF environment, the optimal policy is always going right, for every single state. Great, but there are still many unanswered questions. For instance, how much reward should I expect from this policy? Because, even though we know how to act optimally, the environment might send our agent backward to the hole even if we always select to go toward the goal. This is why returns aren't enough. The agent is really looking to maximize the expected return; that means the return taking into account the environment's stochasticity.

Also, we need a method to automatically find optimal policies, because in the FL example, for instance, it isn't obvious at all what the optimal policy looks like!

There are a few components that are kept internal to the agent that can help it find optimal behavior: there are policies, there can be multiple policies for a given environment, and in fact, in certain environments, there may be multiple optimal policies. Also, there are value functions to help us keep track of return estimates. There's a single optimal value function for a given MDP, but there may be multiple value functions in general.

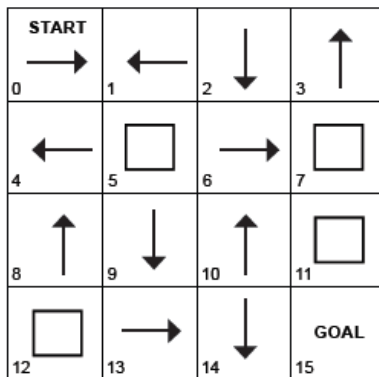
Optimal policy in the SWF environment



Policies

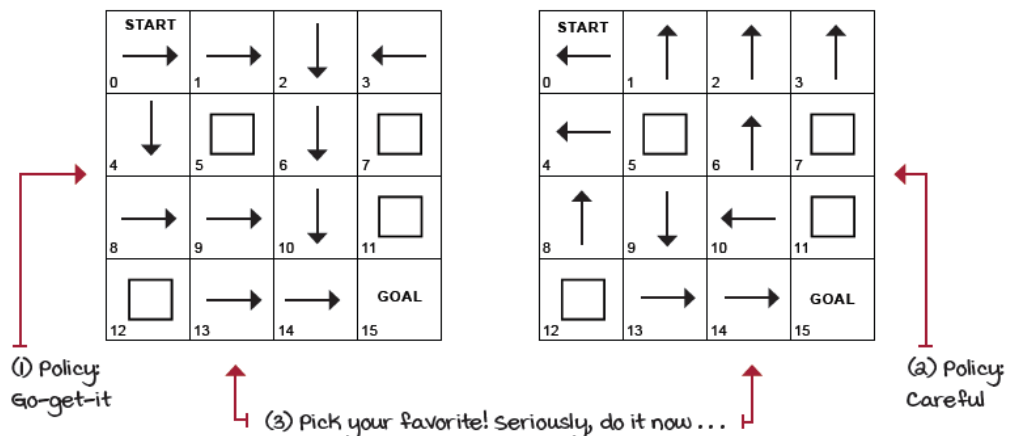
Given the stochasticity in the Frozen Lake environment (and most reinforcement learning problems,) the agent needs to find a policy, denoted as π . A policy is a function that prescribes actions to take for a given nonterminal state. (Remember, policies can be stochastic: either directly over an action or a probability distribution over actions. Here's a sample policy.

A randomly generated policy



(i) A policy generated uniformly at random. Nothing special so far...

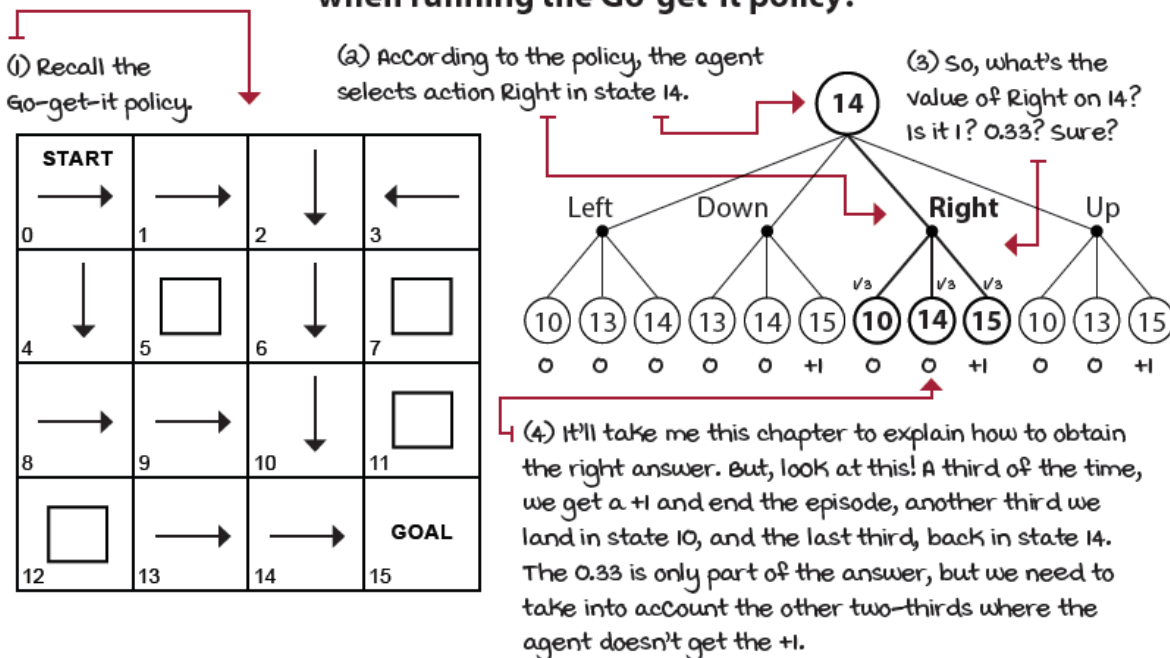
One immediate question that arises when looking at a policy is this: how good is this policy? If we find a way to put a number to policies, we could also ask the question, how much better is this policy compared to another policy?

How can we compare policies?**State-value function:**

Something that'd help us compare policies is to put numbers to states for a given policy. That is, if we're given a policy and the MDP, we should be able to calculate the expected return starting from every single state (we care mostly about the START state). How can we calculate how valuable being in a state is? For instance, if our agent is in state 14 (to the left of the GOAL), how is that better than being in state 13 (to the left of 14)? And precisely how much better is it? More importantly, under which policy would we have better results, the Go-get-it or the Careful policy?

Let's give it a quick try with the Go-get-it policy. What is the value of being in state 14 under the Go-get-it policy?

What's the value of being in state 14 when running the Go-get-it policy?



We now define the value of a state s when following a policy π : the value of a state s under policy π is the expectation of returns if the agent follows policy π starting from state s . Calculate this for every state, and you get the state-value function, or V-function or value function. It represents the expected return when following policy π from state s .

(1) The value of a state s ... (3) ... is the expectation over π ...

(2) ... under policy π ... $v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$ (5) ... given you select state s at time step t .

(4) ... of returns at time step t ...

(6) Remember that returns are the sum of discounted rewards. $v_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots | S_t = s]$

(7) And we can define them recursively like so. $v_{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} | S_t = s]$

→ (8) This equation is called the Bellman equation, and it tells us how to find the value of states.

→
$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')], \forall s \in S$$

(9) We get the action (or actions, if the policy is stochastic) prescribed for state s and do a weighted sum.

(10) We also weight the sum over the probability of next states and rewards.

(11) We add the reward and the discounted value of the landing state, then weight that by the probability of that transition occurring.

(12) Do this for all states in the state space.

These equations are fascinating. A bit of a mess given the recursive dependencies, but still interesting. Notice how the value of a state depends recursively on the value of possibly many other states, which values may also depend on others, including the original state. Note that the state-value function is often referred to as the value function, or even the V-function, or more simply $V^{\pi}(s)$.

Action-value function

Another critical question that we often need to ask isn't merely about the value of a state but the value of taking action a in a state s . Differentiating answers to this kind of question will help us decide between actions. By comparing between different actions under the same policy, we can select better actions, and therefore improve our policies. The *action-value function*, also known as *Q-function* or $Q^{\pi}(s, a)$, captures precisely this: the expected return if the agent follows policy π after taking action a in state s .

The Q-function, on the other hand, does somewhat capture the dynamics of the environment and allows you to improve policies without the need for MDPs.

(1) The value of action a in state s under policy $\pi \dots$

(2) $\dots$ is the expectation of returns, given we select action a in state s and follow policy π thereafter.

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

(3) We can define this equation recursively like so:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[R_t + \gamma G_{t+1} | S_t = s, A_t = a]$$

(4) The Bellman equation for action values is defined as follows:

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')], \forall s \in S, \forall a \in A(s)$$

(5) Notice we don't weigh over actions because we're interested only in a specific action.

(6) We do weigh, however, by the probabilities of next states and rewards.

(7) What do we weigh? The sum of the reward and the discounted value of the next state.

(8) We do that for all state-action pairs.